Configuración de entornos virtuales con Conda

Edison Achalma
Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga

Resumen

Esta publicación presenta una guía práctica y detallada para la creación, gestión y mantenimiento de entornos virtuales aislados utilizando Conda (y Mamba como acelerador), orientada especialmente a economistas, investigadores y científicos de datos que manejan múltiples proyectos académicos y de automatización. Se explica la filosofía detrás de los entornos separados, se propone una estrategia concreta de entornos por propósito (econblog, scripts-env, datascience, teaching), y se documentan paso a paso; creación de entornos específicos, selección cuidadosa de paquetes (tanto de conda-forge como vía pip), generación y mantenimiento de archivos environment.yml reproducibles, integración óptima con JupyterLab, VS Code y Quarto, configuración de alias para flujos de trabajo eficientes, mejores prácticas para colaboración y reproducibilidad, y resolución de problemas frecuentes. El enfoque prioriza evitar conflictos de dependencias, garantizar la reproducibilidad de análisis econométricos y publicaciones académicas, y mantener un sistema ligero y organizado.

Palabras Claves: Entornos virtuales, Conda, Miniconda

Tabla de contenidos

Introduction **4**

1 Guía completa de configuración de entornos virtuales con Conda **4**
1.1 Tabla de contenidos 4
1.2 1. Conceptos fundamentales: bibliotecas y repositorios 5
1.3 2. Filosofía de entornos separados 5
1.3.1 2.1. Principio fundamental: nunca instalar paquetes en `base` 5
1.3.2 2.2. Un entorno por propósito, no por proyecto individual 5
1.4 3. Creación de entornos 6

Edison Achalma https://orcid.org/0000-0001-6996-3364

El autor no tiene conflictos de interés que revelar. Los roles de autor se clasificaron utilizando la taxonomía de roles de colaborador (CRediT; https://credit.niso.org/) de la siguiente manera: Edison Achalma: conceptualización, metodología, análisis formal, investigación, recursos, curación de datos, redacción, visualización, supervisión, administración del proyecto

La correspondencia relativa a este artículo debe dirigirse a Edison Achalma, Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga, Portal Independencia N° 57, Ayacucho, AYA 5001, Perú, Email: elmer.achalma.09@unsch.edu.pe

1.4.1 3.1. Crear un entorno vacío . . . 6
1.4.2 3.2. Crear un entorno con una versión específica de Python . . . 6
1.4.3 3.3. Crear un entorno con un paquete específico . . . 6
1.4.4 3.4. Crear un entorno con una versión específica de un paquete . . . 6
1.4.5 3.5. Crear un entorno con Python y múltiples paquetes a la vez . . . 6
1.4.6 3.6. Paquetes por defecto en cada entorno nuevo . . . 6
1.5 4. Creación de entornos desde `environment.yml` . . . 7
1.5.1 4.1. Estructura básica de un `environment.yml` . . . 7
1.5.2 4.2. Crear el entorno desde el archivo . . . 7
1.5.3 4.3. Activar y verificar . . . 7
1.6 5. Ubicación de entornos: por nombre vs. por prefijo . . . 7
1.6.1 5.1. Entornos por nombre (comportamiento por defecto) . . . 7
1.6.2 5.2. Entornos por prefijo (ubicación personalizada) . . . 7
1.7 6. Plataforma de destino de un entorno . . . 8
1.8 7. Activación y desactivación . . . 9
1.8.1 7.1. Qué hace realmente la activación . . . 9
1.8.2 7.2. Activar un entorno . . . 9
1.8.3 7.3. Desactivar un entorno . . . 9
1.8.4 7.4. Determinar tu entorno activo actual . . . 10
1.9 8. `conda init` e integración con shells . . . 10
1.9.1 8.1. Inicialización mínima (`--condabin`) . . . 10
1.9.2 8.2. Activación automática del entorno por defecto . . . 10
1.10 9. Activación anidada (*stacking*) . . . 11
1.11 10. Gestión de canales . . . 11
1.11.1 10.1. Qué son y cómo se resuelven colisiones . . . 11
1.11.2 10.2. Agregar un canal . . . 11
1.11.3 10.3. Prioridad estricta de canales (recomendada) . . . 12
1.11.4 10.4. Canal por defecto de tu proyecto: `conda-forge` . . . 12
1.12 11. Gestión de paquetes dentro de un entorno . . . 12
1.12.1 11.1. Listar paquetes instalados . . . 12
1.12.2 11.2. Instalar paquetes . . . 12
1.12.3 11.3. Actualizar paquetes . . . 13
1.12.4 11.4. Buscar paquetes . . . 13
1.13 12. Conda y pip: cómo combinarlos correctamente . . . 13
1.13.1 12.1. Instalar pip dentro de un entorno conda . . . 13
1.13.2 12.2. Reglas oficiales para combinar conda y pip . . . 13
1.13.3 12.3. Orden correcto en la práctica . . . 13
1.14 13. Actualizar un entorno existente . . . 14
1.15 14. Congelar un entorno (frozen environments) . . . 14
1.16 15. Clonar un entorno . . . 14
1.17 16. Exportar entornos: formatos y estrategias . . . 15
1.17.1 16.1. El comando moderno `conda export` (conda 25.7.x en adelante) . . . 15
1.17.2 16.2. Compatibilidad multiplataforma con `--from-history` . . . 16
1.17.3 16.3. El comando tradicional `conda env export` . . . 16
1.17.4 16.4. Crear un `environment.yml` manualmente . . . 16
1.18 17. Construir entornos idénticos con especificaciones explícitas . . . 17
1.18.1 17.1. Generar la especificación explícita . . . 17
1.18.2 17.2. Usar el archivo para crear un entorno idéntico . . . 18

1.18.3 17.3. Generar lockfiles explícitos sin crear un entorno 18
1.19 18. Restaurar un entorno a una revisión anterior 18
1.20 19. Eliminar entornos 19
1.21 20. Variables de entorno asociadas a un entorno conda 19
1.21.1 20.1. Vía la API de configuración (recomendado) 19
1.21.2 20.2. Vía scripts de activación/desactivación (avanzado) 20
1.22 21. Estrategia de entornos por propósito para tu flujo de trabajo 20
1.22.1 21.1. Mapa de entornos 21
1.22.2 21.2. Matriz de decisión 21
1.22.3 21.3. Entorno `econblog` 21
1.22.4 21.4. Entorno `scripts-env` 22
1.22.5 21.5. Entorno `datascience` 22
1.22.6 21.6. Entorno `teaching` 23
1.23 22. `environment.yml` de referencia para cada entorno 23
1.23.1 22.1. `econblog/environment.yml` 23
1.23.2 22.2. `scripts-env/environment.yml` 24
1.23.3 22.3. `datascience/environment.yml` 25
1.24 23. Reproducibilidad y colaboración vía Git 26
1.24.1 23.1. Replicar un entorno en otra máquina 26
1.24.2 23.2. Compartir con colaboradores: instrucciones en `README.md` 26
1.24.3 23.3. Actualizar un entorno existente tras cambios en `environment.yml` 26
1.25 24. Integración con Jupyter Lab 27
1.25.1 24.1. Jupyter debe instalarse en cada entorno que lo use 27
1.25.2 24.2. Registrar un kernel para un entorno 27
1.25.3 24.3. Iniciar Jupyter Lab desde un entorno específico 27
1.26 25. Integración con VS Code 27
1.26.1 25.1. Seleccionar el intérprete de conda 27
1.26.2 25.2. Configurar el workspace 27
1.26.3 25.3. Verificar activación en la terminal integrada 27
1.27 26. Mamba como acelerador 28
1.28 27. Resolución de problemas comunes 28
1.28.1 27.1. “Kernel died” en Jupyter 28
1.28.2 27.2. `ModuleNotFoundError` en Jupyter pese a tener el paquete instalado 28
1.28.3 27.3. Conflictos de versiones entre paquetes 29
1.28.4 27.4. Entorno demasiado grande en disco 29
1.29 28. Referencia rápida de comandos 29
1.30 39. Referencias oficiales 30

2 Publicaciones Similares 31

Configuración de entornos virtuales con Conda

1 Guía completa de configuración de entornos virtuales con Conda

Referencia técnica independiente sobre la **configuración y gestión de entornos virtuales** con conda (y mamba como acelerador), aplicable en Kubuntu/Ubuntu, Arch Linux y Windows. Basada en la documentación oficial de **conda** (docs.conda.io) y organizada siguiendo la misma estrategia de entornos por propósito que ya usas en tu flujo de trabajo (econblog, scripts-env, datascience, teaching), documentada previamente en tu publicación "Entornos virtuales con Conda" (numerus-scriptum.netlify.app). Esta guía asume que conda/mamba ya están instalados; para instalación, consulta la guía dedicada a ese tema.

1.1 Tabla de contenidos

1. Conceptos fundamentales: bibliotecas y repositorios
2. Filosofía de entornos separados
3. Creación de entornos
4. Creación de entornos desde `environment.yml`
5. Ubicación de entornos: por nombre vs. por prefijo
6. Plataforma de destino de un entorno
7. Activación y desactivación
8. `conda init` e integración con shells
9. Activación anidada (*stacking*)
10. Gestión de canales
11. Gestión de paquetes dentro de un entorno
12. Conda y pip: cómo combinarlos correctamente
13. Actualizar un entorno existente
14. Congelar un entorno (frozen environments)
15. Clonar un entorno
16. Exportar entornos: formatos y estrategias
17. Construir entornos idénticos con especificaciones explícitas
18. Restaurar un entorno a una revisión anterior
19. Eliminar entornos
20. Variables de entorno asociadas a un entorno conda
21. Estrategia de entornos por propósito para tu flujo de trabajo
22. `environment.yml` de referencia para cada entorno
23. Reproducibilidad y colaboración vía Git
24. Integración con Jupyter Lab
25. Integración con VS Code
26. Mamba como acelerador
27. Resolución de problemas comunes
28. Checklist de buenas prácticas
29. Referencia rápida de comandos
30. Referencias oficiales

1.2 1. Conceptos fundamentales: bibliotecas y repositorios

Antes de gestionar entornos, conviene fijar dos términos que la documentación oficial de conda distingue:

- **Canal (*channel*)**: una ubicación (típicamente una URL) donde se almacenan paquetes conda. El comando `conda` busca, por defecto, en un conjunto predeterminado de canales, y los paquetes se descargan y actualizan automáticamente desde el canal por defecto (`repo.anaconda.com/pkgs/`). El canal comunitario más usado para ciencia de datos es `conda-forge`.
- **Entorno (*environment*)**: una biblioteca de paquetes aislada y autocontenida, con su propia versión de Python y su propio conjunto de paquetes, independiente de cualquier otro entorno o de la biblioteca de sistema.

1.3 2. Filosofía de entornos separados

El problema sin entornos: sin aislamiento, todos los paquetes comparten el mismo espacio. Esto provoca conflictos de versiones (por ejemplo `pandas 1.5` vs `pandas 2.0` requeridos por proyectos distintos), dependencias rotas al actualizar, imposibilidad de reproducir análisis antiguos, y proyectos que interfieren entre sí.

La solución: un entorno aislado por proyecto o por tipo de trabajo.

1.3.1 2.1. Principio fundamental: nunca instalar paquetes en `base`

El entorno `base` es el que conda activa por defecto al inicializar la shell. Instalar paquetes de proyecto directamente ahí termina, con el tiempo, recreando el mismo problema que los entornos buscan evitar: un único espacio compartido y contaminado.

```
# INCORRECTO
conda activate base
mamba install pandas   # NO HACER ESTO

# CORRECTO
conda activate mi-proyecto
mamba install pandas   # instalar en el entorno específico
```

`base` debería contener, como máximo, conda/mamba mismos y herramientas de gestión de entornos; nada de las dependencias de un proyecto particular.

1.3.2 2.2. Un entorno por propósito, no por proyecto individual

No es necesario (ni conveniente) crear un entorno nuevo por cada repositorio. La estrategia más sostenible es agrupar proyectos que comparten el mismo conjunto de dependencias bajo un mismo entorno con nombre temático (por ejemplo `econblog`, `scripts-env`), reservando entornos dedicados para casos con requisitos claramente distintos (paquetes pesados, versiones fijas para estudiantes, etc.). Esto se desarrolla con detalle en la sección 21.

1.4 3. Creación de entornos

1.4.1 3.1. Crear un entorno vacío

```
conda create --name mi-entorno
```

Cuando conda pregunte si proceder, escribe y. Esto crea el entorno en el directorio `envs/` de tu instalación, sin paquetes instalados.

1.4.2 3.2. Crear un entorno con una versión específica de Python

```
conda create -n myenv python=3.9
```

1.4.3 3.3. Crear un entorno con un paquete específico

```
conda create -n myenv scipy
```

o, equivalentemente, en dos pasos:

```
conda create -n myenv python
conda install -n myenv scipy
```

1.4.4 3.4. Crear un entorno con una versión específica de un paquete

```
conda create -n myenv scipy=0.17.3
```

1.4.5 3.5. Crear un entorno con Python y múltiples paquetes a la vez

```
conda create -n myenv python=3.9 scipy=0.17.3 astroid babel
```

> **Recomendación oficial importante**: instala todos los programas que quieras en un entorno **al mismo tiempo**. Instalarlos uno por uno, en comandos separados, puede provocar conflictos de dependencias que un único comando de resolución conjunta evitaría.

1.4.6 3.6. Paquetes por defecto en cada entorno nuevo

Para que conda instale automáticamente pip (u otro programa) cada vez que creas un entorno nuevo, agrega los paquetes por defecto a la sección `create_default_packages` de tu archivo `.condarc`. Si en un caso puntual no deseas esos paquetes por defecto:

```
conda create --no-default-packages -n myenv python
```

1.5 4. Creación de entornos desde `environment.yml`

Este es el método recomendado para entornos reproducibles y compartibles, y el que usarás en la práctica para tus proyectos (ver sección 22).

1.5.1 4.1. Estructura básica de un `environment.yml`

```
# environment.yml
name: myenv
channels:
  - conda-forge
dependencies:
  - python
  - numpy
```

1.5.2 4.2. Crear el entorno desde el archivo

```
conda create --file environment.yml
```

El nombre del entorno se toma del campo `name:` dentro del propio archivo.

> **Nota oficial sobre sintaxis histórica**: crear entornos desde archivos YAML solía hacerse con `conda env create --file <archivo>`. Esta forma **sigue funcionando** y se mantiene por compatibilidad con scripts y automatizaciones existentes, pero `conda create --file` es ahora la forma preferida según la documentación oficial.

1.5.3 4.3. Activar y verificar

```
conda activate myenv
conda env list
```

(también puedes usar `conda info --envs`, equivalente).

1.6 5. Ubicación de entornos: por nombre vs. por prefijo

1.6.1 5.1. Entornos por nombre (comportamiento por defecto)

Por defecto, los entornos creados con `--name`/`-n` viven en la carpeta `envs/` de tu instalación de conda (por ejemplo `~/miniconda3/envs/`).

1.6.2 5.2. Entornos por prefijo (ubicación personalizada)

Puedes controlar dónde vive un entorno indicando una ruta de destino con `--prefix`/`-p`:

```
conda create --prefix ./envs jupyterlab=3.2 matplotlib=3.5 numpy=1.21
```

Se activa igual que un entorno por nombre, pero usando la ruta:

```
conda activate ./envs
```

Ventajas documentadas oficialmente de usar `--prefix` dentro de la carpeta del proyecto:

- Es fácil saber, con solo ver la estructura de carpetas, si tu proyecto usa un entorno aislado (aparece como subcarpeta).
- El proyecto queda más autocontenido: todo, incluido el software requerido, vive dentro de un único directorio de proyecto.
- Te permite reutilizar el mismo nombre de subcarpeta (por ejemplo `envs` o `env`) en todos tus proyectos, en vez de tener que inventar un nombre distinto para cada entorno.

Consideraciones a tener en cuenta si usas `--prefix`:

1. Conda ya no puede encontrar tu entorno con la bandera `--name`; generalmente necesitarás pasar `--prefix` junto con la ruta completa del entorno.
2. Tu prompt mostrará la ruta absoluta completa del entorno activo en vez de su nombre, lo cual puede resultar en prefijos largos en la terminal, por ejemplo:

```
(/home/achalmaedison/Proyectos/mi-proyecto/envs) $
```

Para evitar ese prefijo largo, ajusta la configuración `env_prompt` en tu `.condarc`:

```
conda config --set env_prompt '({name})'
```

Esto edita (o crea) tu archivo `.condarc`, y a partir de ahí el prompt mostrará solo el nombre genérico del entorno (el nombre de su carpeta raíz):

```
$ cd project-directory
$ conda activate ./env
(env) project-directory $
```

1.7 6. Plataforma de destino de un entorno

Por defecto, conda crea entornos dirigidos a la plataforma sobre la que se está ejecutando. Puedes consultar tu plataforma actual con `conda info` (campo `platform`).

En casos avanzados, puedes especificar una plataforma distinta con la bandera `--platform`, disponible en `conda create` y `conda env create`. Por ejemplo, para crear un entorno orientado a procesadores Intel desde una Mac con Apple Silicon (emulado vía Rosetta):

```
conda create --platform osx-64 --name python-x64 python
```

> **Nota oficial**: no se puede especificar `--platform` para entornos ya existentes; una vez creado, el entorno queda anotado con esa configuración y las operaciones posteriores recuerdan la plataforma de destino. La documentación oficial también advierte que especificar un sistema operativo distinto (por ejemplo, crear un entorno Linux desde macOS) no se recomienda fuera de operaciones `--dry-run`, ya que suele fallar por paquetes virtuales faltantes o por incompatibilidades de sistema de archivos.

1.8 7. Activación y desactivación

1.8.1 7.1. Qué hace realmente la activación

Activar un entorno cumple dos funciones principales: añade entradas al `PATH` correspondientes al entorno, y ejecuta cualquier script de activación que el entorno pueda contener (estos scripts son cómo los paquetes configuran variables de entorno arbitrarias necesarias para su funcionamiento). La activación antepone rutas al `PATH` solo mientras el entorno está activo en esa sesión de terminal; no es un cambio global del sistema.

> **Advertencia oficial importante**: durante la instalación de Anaconda/Miniconda existe la opción "Add Anaconda to my PATH environment variable" — **no se recomienda usarla**, porque *añade* (append) Anaconda al PATH sin llamar a los scripts de activación correspondientes. La forma correcta de integrar conda con tu shell es `conda init` (ver sección 8), no esa casilla del instalador.

1.8.2 7.2. Activar un entorno

```
conda activate myenv
```

(sustituye `myenv` por el nombre del entorno, o por la ruta si lo creaste con `--prefix`).

Si ves una advertencia indicando que el intérprete de Python está en un entorno conda pero el entorno no fue activado correctamente, significa que necesitas activar el entorno apropiadamente (en Windows, ejecutando `c:\Anaconda3\Scripts\activate` si es necesario).

> **Nota oficial sobre Windows**: Windows es especialmente sensible a una activación correcta, porque su cargador de bibliotecas no soporta el concepto de bibliotecas y ejecutables que saben dónde buscar sus propias dependencias (RPATH, como sí ocurre en Linux/macOS). Windows depende en cambio de un orden de búsqueda de bibliotecas de enlace dinámico. Si los entornos no están activos correctamente, las bibliotecas no se encontrarán y aparecerán muchos errores — los errores de HTTP o SSL son comunes cuando el Python de un entorno hijo no puede encontrar la biblioteca OpenSSL necesaria.

1.8.3 7.3. Desactivar un entorno

```
conda deactivate
```

> **Nota oficial**: para simplemente volver al entorno por defecto, es mejor llamar a `conda activate` sin especificar ningún entorno, en vez de intentar desactivar repetidamente. Si ejecutas `conda deactivate` estando en tu entorno `base`, podrías perder la capacidad de ejecutar `conda` en esa shell (no es grave: es local a esa sesión, basta con abrir una nueva terminal). Sin embargo, si el entorno fue activado con `--stack` (o se apiló automáticamente), sí es preferible usar `conda deactivate` para deshacerlo correctamente.

1.8.4 7.4. Determinar tu entorno activo actual

Por defecto, el entorno activo se muestra entre paréntesis `()` o corchetes `[]` al inicio de tu prompt:

```
(myenv) $
```

Si no ves esto, ejecuta `conda info --envs`; tu entorno actual aparecerá marcado con un asterisco (`*`).

Para deshabilitar/habilitar que el nombre del entorno aparezca en el prompt:

```
conda config --set changeps1 false   # deshabilitar
conda config --set changeps1 true    # rehabilitar
```

1.9 8. `conda init` e integración con shells

Versiones tempranas de conda requerían pasos manuales adicionales para que `conda activate` funcionara correctamente. Conda 4.4 introdujo `conda activate`, y conda 4.6 añadió soporte de inicialización extensivo para que conda funcione de forma más rápida y menos disruptiva en una amplia variedad de shells: **bash, zsh, csh, fish, xonsh, y más**. Esto es lo que ejecuta el instalador cuando aceptas inicializar conda (ver guía de instalación).

> **Relevante para tu configuración (zsh + kitty)**: dado que tu shell por defecto es `zsh`, `conda init` detecta y modifica `~/.zshrc` automáticamente. Si en algún contexto necesitas integrar conda con `fish` además de `zsh`, puedes ejecutar `conda init fish` adicionalmente; ambas inicializaciones pueden coexistir.

1.9.1 8.1. Inicialización mínima (`--condabin`)

Si prefieres no instalar una función de shell completa en tu perfil, `conda init --condabin` añade únicamente el directorio `$CONDA_PREFIX/condabin/` al `PATH`. Esa carpeta solo contiene el ejecutable `conda`, por lo que es mínimamente invasiva.

1.9.2 8.2. Activación automática del entorno por defecto

El ajuste `auto_activate` (booleano) controla si conda activa automáticamente el entorno por defecto cada vez que arranca tu shell. El comando `conda` estará disponible de cualquier forma, pero sin activar el entorno, ningún otro programa del entorno por defecto estará disponible hasta que lo actives explícitamente con `conda activate`.

El entorno a activar por defecto se configura con:

```
default_activation_env: str
```

> Algunas personas desactivan `auto_activate` para acelerar el arranque de su shell, o para evitar que software instalado vía conda oculte automáticamente otro software del sistema.

1.10 9. Activación anidada (*stacking*)

Por defecto, `conda activate` **desactiva** el entorno actual antes de activar el nuevo, y lo reactiva al desactivar el nuevo. A veces interesa conservar las entradas de `PATH` del entorno actual para seguir teniendo acceso fácil a programas de línea de comandos de ese primer entorno (común cuando ciertas utilidades están instaladas en el entorno por defecto).

Para conservar el entorno actual en el `PATH` al activar uno nuevo:

```
conda activate --stack myenv
```

Para que esto ocurra automáticamente siempre que actives un entorno desde el entorno más externo (típicamente el entorno por defecto):

```
conda config --set auto_stack 1
```

Puedes especificar un número mayor para permitir niveles más profundos de apilamiento automático, aunque la documentación **no lo recomienda**, ya que niveles más profundos de stacking tienden a generar confusión.

1.11 10. Gestión de canales

1.11.1 10.1. Qué son y cómo se resuelven colisiones

Los canales son las ubicaciones donde se almacenan los paquetes. Distintos canales pueden tener el mismo paquete, por lo que conda debe resolver estas colisiones. No hay colisiones si usas únicamente el canal por defecto, ni si todos tus canales contienen paquetes que no se solapan entre sí.

Cuando hay colisión, conda procesa los paquetes con el mismo nombre across todos los canales listados de la siguiente forma:

1. Ordena los paquetes de mayor a menor prioridad de canal.
2. Entre paquetes empatados en prioridad de canal, ordena de mayor a menor número de versión.
3. Entre paquetes aún empatados (mismo canal, misma versión), ordena de mayor a menor número de build.
4. Instala el primer paquete de la lista ordenada que satisface las especificaciones de instalación.

1.11.2 10.2. Agregar un canal

Para añadir un canal al principio de la lista (máxima prioridad):

```
conda config --add channels new_channel
```

(equivalente explícito: `conda config --prepend channels new_channel`).

Para añadirlo al final (mínima prioridad):

```
conda config --append channels new_channel
```

1.11.3 10.3. Prioridad estricta de canales (recomendada)

Desde la versión 4.6.0, conda ofrece la función de **prioridad estricta de canales**, que puede acelerar dramáticamente las operaciones de conda y reducir problemas de incompatibilidad entre paquetes. La documentación oficial **recomienda activar `strict` cuando sea posible**:

```
conda config --set channel_priority strict
```

Valores posibles de `channel_priority`:

- `strict`: los paquetes en canales de menor prioridad ni siquiera se consideran si el mismo paquete existe en un canal de mayor prioridad.
- `flexible` (valor por defecto): el solver puede buscar en canales de menor prioridad para satisfacer dependencias, en vez de fallar con un error de no-satisfacibilidad.
- `disabled`: la versión del paquete tiene precedencia; la prioridad de canal configurada solo se usa para desempatar.

1.11.4 10.4. Canal por defecto de tu proyecto: `conda-forge`

Siguiendo la convención que ya usas en tus `environment.yml` (econblog, scripts-env, etc.), `conda-forge` es el canal comunitario estándar para ciencia de datos en el ecosistema conda, con una cobertura de paquetes mucho más amplia que el canal `defaults` de Anaconda Inc. Puedes especificar un canal puntual para un solo paquete dentro de un `environment.yml` con la sintaxis `canal::paquete`, sin necesidad de que ese canal esté en la lista `channels:` general (ver sección 22 para ejemplos completos).

1.12 11. Gestión de paquetes dentro de un entorno

1.12.1 11.1. Listar paquetes instalados

Entorno activado:

```
conda list
```

Entorno no activado, especificándolo por nombre:

```
conda list -n myenv
```

Verificar si un paquete específico está instalado:

```
conda list -n myenv scipy
```

1.12.2 11.2. Instalar paquetes

```
conda install paquete
conda install paquete=1.0.0          # versión específica
conda install -c conda-forge paquete  # desde un canal específico
```

1.12.3 11.3. Actualizar paquetes

```
conda update paquete
conda update --all
```

1.12.4 11.4. Buscar paquetes

```
conda search paquete
conda search paquete --info   # información detallada
```

1.13 12. Conda y pip: cómo combinarlos correctamente

Es habitual necesitar paquetes que solo existen en PyPI (no en conda-forge ni en el canal por defecto). La documentación oficial es explícita sobre el orden y la disciplina necesarios para que esto no genere conflictos.

1.13.1 12.1. Instalar pip dentro de un entorno conda

```
conda install -n myenv pip
conda activate myenv
pip <subcomando-de-pip>
```

1.13.2 12.2. Reglas oficiales para combinar conda y pip

Usa pip solo después de conda: - Instala tantos requisitos como sea posible con conda, y usa pip solo para lo que falte. - pip debería ejecutarse con `--upgrade-strategy only-if-needed` (que es, de hecho, el comportamiento por defecto). - No uses pip con el argumento `--user`; evita instalaciones "para todos los usuarios".

Usa entornos conda para aislamiento: - Crea un entorno conda específicamente para aislar cualquier cambio que haga pip. - Los entornos ocupan poco espacio adicional gracias a los *hard links* que usa conda internamente. - Ten cuidado de no ejecutar pip en el entorno raíz/`base`.

Recrea el entorno si necesitas hacer cambios: - Una vez que pip ha sido usado, conda **ya no es consciente** de esos cambios. - Para instalar paquetes conda adicionales después de haber usado pip, lo más seguro es **recrear el entorno** desde cero, en vez de simplemente correr conda de nuevo encima.

Guarda los requisitos de conda y pip en archivos de texto: - Los requisitos de conda pueden pasarse vía el argumento `--file`. - pip acepta una lista de paquetes Python con `-r`/`--requirements`. - `conda env` exporta o crea entornos basados en un archivo combinado de requisitos conda y pip (ver sección 22).

1.13.3 12.3. Orden correcto en la práctica

```
mamba install pandas matplotlib -y   # 1. primero conda/mamba
pip install wooldridge               # 2. luego pip, solo lo que falte

# NUNCA mezclar en el mismo comando o cadena:
# conda install pandas && pip install wooldridge   # evitar este patrón
```

1.14 13. Actualizar un entorno existente

Puede que necesites actualizar tu entorno por varias razones: una dependencia central lanzó una nueva versión, necesitas un paquete adicional para un análisis, o encontraste un paquete mejor y ya no necesitas el anterior.

En cualquiera de estos casos, actualiza el contenido de tu `environment.yml` y luego ejecuta:

```
conda env update --file environment.yml --prune
```

> **Nota oficial sobre `--prune`**: esta opción hace que conda elimine del entorno cualquier dependencia que ya no sea requerida según el archivo actualizado. Sin `--prune`, los paquetes removidos del `environment.yml` permanecerían instalados aunque ya no estén declarados.

1.15 14. Congelar un entorno (frozen environments)

Una función relativamente reciente del estándar conda (CEP 22) introduce un archivo marcador de entorno que indica a conda que **no permita modificaciones** en un entorno dado.

Al intentar añadir, actualizar o eliminar un paquete en un entorno congelado, obtendrás por defecto un error:

```
EnvironmentIsFrozenError: Cannot not modify '~/.conda/envs/my-env'.
The environment is marked as frozen. You can ignore this error with
the `--override-frozen` flag, at your own risk.
```

Es posible pasar la bandera `--override-frozen` para forzar la modificación, pero la documentación oficial **no lo recomienda**: solo debería usarse por usuarios avanzados conscientes de los riesgos y capaces de resolver las posibles complicaciones derivadas de esa operación.

> **Caso de uso relevante para ti**: esto encaja con tu entorno `teaching`, donde quieres versiones estables y probadas para tus estudiantes; congelar ese entorno una vez validado evita modificaciones accidentales que rompan el material de clase a mitad de semestre.

1.16 15. Clonar un entorno

Para hacer una copia exacta de un entorno existente:

```
conda create --name myclone --clone myenv
```

(sustituye `myclone` por el nombre del nuevo entorno, y `myenv` por el entorno existente a copiar).

Verificar que la copia se haya creado:

```
conda info --envs
```

Deberías ver ambos entornos —el original y la copia nueva— en la lista.

Uso práctico: clonar es ideal para experimentar con cambios riesgosos (por ejemplo, probar una actualización mayor de `pandas` o `statsmodels`) sin arriesgar tu entorno de producción; si el experimento sale mal, simplemente eliminas el clon.

1.17 16. Exportar entornos: formatos y estrategias

1.17.1 16.1. El comando moderno `conda export` (conda 25.7.x en adelante)

Según la documentación oficial, `conda export` fue significativamente mejorado con una arquitectura basada en plugins, soporte de múltiples formatos de salida, y funcionalidad ampliada.

Formatos disponibles:

Formato	Descripción
`environment-yaml`	Formato YAML multiplataforma (por defecto)
`environment-json`	Formato JSON multiplataforma
`explicit`	URLs explícitas específicas de la plataforma (compatible con CEP 23)
`requirements`	Formato de requisitos específico de la plataforma

Uso básico:

```
conda export > environment.yaml                          # YAML por defecto
conda export --name myenv --format=environment-yaml       # entorno específico
conda export --file=environment.yaml                       # detección automática por
conda export --file=explicit.txt                           # detecta formato explícito
conda export --file=requirements.txt                        # detecta formato requirem
```

Formato explícito (reproducción exacta en la misma plataforma):

```
conda export --format=explicit --file=explicit.txt
```

Genera URLs completas de paquetes:

```
@EXPLICIT
https://repo.anaconda.com/pkgs/main/osx-64/python-3.9.7-
h88f2d9e_0.tar.bz2
https://repo.anaconda.com/pkgs/main/osx-64/numpy-1.21.0-
py39h2e5f516_0.tar.bz2
```

1.17.2 16.2. Compatibilidad multiplataforma con `--from-history`

Para máxima portabilidad entre sistemas operativos (por ejemplo, exportar desde Kubuntu y restaurar en Windows), usa `--from-history`:

```
conda export --from-history --format=environment-yaml
```

Esto exporta **únicamente los paquetes que instalaste explícitamente**, excluyendo dependencias específicas de plataforma que se resolvieron automáticamente. Si creas un entorno e instalas, por ejemplo, `python=3.7` y `codecov`, conda descargará e instalará numerosos paquetes adicionales para resolver dependencias; muchos de esos paquetes auxiliares podrían no ser compatibles entre plataformas. `--from-history` exporta solo lo que tú elegiste explícitamente:

```
name: env-name
channels:
  - conda-forge
  - defaults
dependencies:
  - python=3.7
  - codecov
prefix: /Users/username/anaconda3/envs/env-name
```

> **Nota oficial**: la bandera `--from-history` solo funciona con formatos estructurados (`environment-yaml`, `environment-json`); no es compatible con los formatos de texto (`explicit`, `requirements`).

1.17.3 16.3. El comando tradicional `conda env export`

También puedes usar el comando tradicional, exclusivo para YAML:

```
conda activate myenv
conda env export > environment.yml
```

> **Advertencia oficial**: si ya tienes un archivo `environment.yml` en tu directorio actual, será **sobrescrito** durante esta operación. Este archivo gestiona tanto los paquetes de pip como los de conda dentro del entorno.

1.17.4 16.4. Crear un `environment.yml` manualmente

También puedes escribir el archivo a mano, sin exportar desde un entorno existente:
Ejemplo simple:

```
name: stats
dependencies:
  - numpy
  - pandas
```

Ejemplo más completo, con canal específico por paquete y sección pip:

```yaml
name: stats2
channels:
  - javascript
dependencies:
  - python=3.9
  - bokeh=2.4.2
  - conda-forge::numpy=1.21.*
  - nodejs=16.13.*
  - flask
  - pip
  - pip:
    - Flask-Testing
```

Notas sobre esta sintaxis (oficiales):

- El uso del comodín `*` (por ejemplo `1.21.*`) mantiene fijas las versiones mayor y menor, permitiendo cualquier número de parche; esto te da actualizaciones de errores (*bugfixes*) sin perder consistencia en el entorno.
- La sintaxis `canal::paquete` permite especificar de qué canal instalar un paquete puntual, sin necesidad de que ese canal aparezca en la lista `channels`: general — útil si quieres que **solo algunos** paquetes (no todos) vengan de un canal comunitario como conda-forge.
- Puedes excluir los canales por defecto agregando `nodefaults` a la lista de canales:

```yaml
channels:
  - javascript
  - nodefaults
```

Esto equivale a pasar `--override-channels` a la mayoría de comandos `conda`. Agregar `nodefaults` en un `environment.yml` afecta solo a ese entorno, mientras que quitar `defaults` de tu `.condarc` afectaría a todos tus entornos.

1.18 17. Construir entornos idénticos con especificaciones explícitas

Para reproducir un entorno de forma exacta en la **misma plataforma de sistema operativo** (misma máquina u otra distinta):

1.18.1 17.1. Generar la especificación explícita

```
conda list --explicit
```

Esto produce una lista como:

```
# This file may be used to create an environment using:
# $ conda create --name <env> --file <this file>
# platform: osx-64
@EXPLICIT
https://repo.anaconda.com/pkgs/free/osx-64/mkl-11.3.3-0.tar.bz2
https://repo.anaconda.com/pkgs/free/osx-64/numpy-1.11.1-py35_0.tar.bz2
...
```

Guardarlo en un archivo:

```
conda list --explicit > spec-file.txt
```

Nota oficial: un archivo de especificación explícita normalmente **no es multiplataforma**; por eso incluye un comentario como `# platform: osx-64` indicando dónde se generó y dónde se sabe que funciona. En otras plataformas, los paquetes listados podrían no estar disponibles, o faltar dependencias para algunos paquetes clave.

1.18.2 17.2. Usar el archivo para crear un entorno idéntico

```
conda create --name myenv --file spec-file.txt
```

O para instalar esos paquetes dentro de un entorno ya existente:

```
conda install --name myenv --file spec-file.txt
```

Conda **no verifica** arquitectura ni dependencias al instalar desde un archivo de especificación. Para asegurar que funcione correctamente, el archivo debe haberse generado desde un entorno funcional, y usarse en la misma arquitectura, sistema operativo y plataforma (por ejemplo `linux-64` o `osx-64`).

1.18.3 17.3. Generar lockfiles explícitos sin crear un entorno

Si solo necesitas el archivo de bloqueo (*lockfile*) sin tener que crear y luego borrar un entorno temporal, puedes invocar conda en modo JSON y procesar la salida con `jq` (necesitas tener `jq` instalado, por ejemplo vía `conda create -n jq jq` o el gestor de paquetes de tu sistema):

Linux/macOS:

```
echo "@EXPLICIT" > explicit.txt
CONDA_PKGS_DIRS=$(mktemp -d) conda create --dry-run MATCHSPECS_AQUI --json | jq -r '.
```

El archivo `explicit.txt` resultante puede usarse para crear un nuevo entorno:

```
conda create -n new-environment --file explicit.txt
```

1.19 18. Restaurar un entorno a una revisión anterior

Conda mantiene un historial de todos los cambios hechos a un entorno, permitiendo "retroceder" fácilmente a una versión previa.

Listar el historial de cambios del entorno actual:

```
conda list --revisions
```

Restaurar a una revisión previa:

```
conda install --revision=REVNUM
```

(o `conda install --rev REVNUM`). Por ejemplo, para volver a la revisión 8:

```
conda install --rev 8
```

Uso práctico: esto es útil como red de seguridad rápida cuando una actualización de paquetes rompe algo y quieres revertir sin reconstruir el entorno entero desde un `environment.yml` de respaldo.

1.20 19. Eliminar entornos

```
conda remove --name myenv --all
```

(equivalente: `conda env remove --name myenv`).
Verificar que el entorno fue eliminado:

```
conda info --envs
```

El entorno eliminado ya no debería aparecer en la lista.

1.21 20. Variables de entorno asociadas a un entorno conda

1.21.1 20.1. Vía la API de configuración (recomendado)

La documentación oficial recomienda este método sobre escribir scripts de activación/desactivación manualmente, ya que estos últimos son ejecución de código arbitrario que puede no ser segura.

```
conda create -n test-env
conda activate test-env
conda env config vars set my_var=value
conda activate test-env   # reactivar para que tome efecto
```

Verificar:

```
echo $my_var                    # %my_var% en Windows
conda env config vars list
```

Eliminar la variable:

```
conda env config vars unset my_var -n test-env
```

Las variables establecidas con `conda env config vars` se conservan en la salida de `conda env export`, y también pueden declararse directamente en el `environment.yml`:

```
name: env-name
channels:
  - conda-forge
  - defaults
dependencies:
  - python=3.7
  - codecov
variables:
  VAR1: valueA
  VAR2: valueB
```

1.21.2 20.2. Vía scripts de activación/desactivación (avanzado)

Si necesitas que un paquete propio o un flujo personalizado configure variables al activar un entorno, puedes crear scripts dedicados.

En Linux/macOS:

```
cd $CONDA_PREFIX
mkdir -p ./etc/conda/activate.d
mkdir -p ./etc/conda/deactivate.d
touch ./etc/conda/activate.d/env_vars.sh
touch ./etc/conda/deactivate.d/env_vars.sh
```

`./etc/conda/activate.d/env_vars.sh`:

```
#!/bin/sh
export MY_KEY='secret-key-value'
export MY_FILE=/path/to/my/file/
```

`./etc/conda/deactivate.d/env_vars.sh`:

```
#!/bin/sh
unset MY_KEY
unset MY_FILE
```

Al ejecutar `conda activate analytics`, las variables se establecen; al ejecutar `conda deactivate`, se eliminan automáticamente.

1.22 21. Estrategia de entornos por propósito para tu flujo de trabajo

Esta sección documenta la estrategia que ya implementaste y mantienes activamente, organizada según la matriz de decisión de tu publicación previa, ahora alineada con la sintaxis y mejores prácticas oficiales descritas arriba.

1.22.1 21.1. Mapa de entornos

```
~/miniconda3/envs/
   econblog/      # Blogs académicos (epsilon-y-beta, axiomata, methodica, etc.)
   scripts-env/   # Scripts de automatización (Quarto, Calibre, Zotero, LibreOffice, L
   datascience/   # Análisis avanzado temporal (paquetes pesados: ML, deep learning)
   teaching/      # Material educativo con versiones fijas y estables
```

1.22.2 21.2. Matriz de decisión

Proyecto / tarea	Entorno	Razón
epsilon-y-beta, axiomata, methodica, aequilibria, optimums, pecunia-fluxus, actus-mercator, res-publica, dialectica-y-mercado, numerus-scriptum	`econblog`	Comparten dependencias: Quarto, pandas, statsmodels
Scripts de Quarto, Calibre, LibreOffice, Linux, Zotero	`scripts-env`	Automatización compartida, dependencias ligeras
Análisis econométrico complejo, machine learning	`datascience`	Paquetes pesados (tensorflow, pytorch, xgboost)
Notebooks para clases	`teaching`	Versiones estables y fijas para estudiantes

1.22.3 21.3. Entorno `econblog`

Creación inicial (instalando todo de una vez, siguiendo la recomendación oficial de la sección 3.5):

```
mamba create -n econblog python=3.12 \
  jupyterlab \
  pandas \
  numpy \
  matplotlib \
  seaborn \
  plotly \
  statsmodels \
  linearmodels \
  pingouin \
  scipy \
  scikit-learn \
  pyarrow \
  openpyxl \
  xlrd \
  -c conda-forge -y
```

Paquetes adicionales de conda-forge:

```
conda activate econblog
mamba install -c conda-forge \
  jupyterlab-lsp \
  myst-parser \
  nbconvert \
  ipywidgets \
  notebook \
  -y
```

Paquetes exclusivos de PyPI (instalando con pip *después* de conda, siguiendo la regla de la sección 12):

```
pip install \
  wooldridge \
  econml \
  causalml \
  upsetplot \
  arch \
  pmdarima
```

1.22.4 21.4. Entorno `scripts-env`

```
mamba create -n scripts-env python=3.12 \
  pyyaml \
  pandas \
  openpyxl \
  python-docx \
  pillow \
  requests \
  beautifulsoup4 \
  lxml \
  -c conda-forge -y

conda activate scripts-env
mamba install -c conda-forge black ruff pytest -y

pip install pypdf2 pdfrw python-frontmatter watchdog
```

1.22.5 21.5. Entorno `datascience`

```
mamba create -n datascience python=3.12 \
  jupyterlab pandas numpy scipy scikit-learn statsmodels \
  xgboost lightgbm matplotlib seaborn plotly bokeh altair networkx \
  -c conda-forge -y

conda activate datascience
pip install tensorflow torch shap optuna linearmodels arch pymc arviz
```

1.22.6 21.6. Entorno `teaching`

```
mamba create -n teaching python=3.11 \
  jupyterlab \
  pandas=2.0 \
  numpy=1.24 \
  matplotlib=3.7 \
  seaborn=0.12 \
  scipy=1.11 \
  statsmodels=0.14 \
  -c conda-forge -y

conda activate teaching
pip install wooldridge jupyter-book jupyterlab-spellchecker
```

> **Candidato natural para congelar (sección 14)**: una vez validado al inicio del semestre, considera marcar este entorno como congelado para evitar modificaciones accidentales mientras lo usan tus estudiantes.

1.23 22. `environment.yml` de referencia para cada entorno

1.23.1 22.1. `econblog/environment.yml`

```
name: econblog

channels:
  - conda-forge
  - defaults

dependencies:
  # Python
  - python=3.12

  # Jupyter y Notebooks
  - jupyterlab>=4.0
  - jupyterlab-lsp
  - ipywidgets
  - notebook
  - nbconvert
  - myst-parser

  # Manipulación de datos
  - pandas>=2.0
  - numpy>=1.24
  - pyarrow
  - openpyxl
```

```
  - xlrd

  # Visualización
  - matplotlib>=3.7
  - seaborn>=0.12
  - plotly>=5.14

  # Estadística y econometría
  - statsmodels>=0.14
  - linearmodels
  - pingouin
  - scipy>=1.11
  - scikit-learn>=1.3

  # Utilidades
  - pip

  # Paquetes exclusivos de PyPI
  - pip:
    - wooldridge
    - econml
    - causalml
    - upsetplot
    - arch
    - pmdarima
```

1.23.2 22.2. scripts-env/environment.yml

```
name: scripts-env

channels:
  - conda-forge
  - defaults

dependencies:
  - python=3.12

  # Manipulación de archivos
  - pyyaml
  - pandas
  - openpyxl
  - python-docx
  - pillow

  # Web scraping y requests
  - requests
  - beautifulsoup4
```

```
  - lxml

  # Desarrollo y testing
  - black
  - ruff
  - pytest

  - pip
  - pip:
    - pypdf2
    - pdfrw
    - python-frontmatter
    - watchdog
```

1.23.3 22.3. datascience/environment.yml

```
name: datascience

channels:
  - conda-forge
  - defaults

dependencies:
  - python=3.12
  - jupyterlab
  - pandas
  - numpy
  - scipy
  - scikit-learn
  - statsmodels
  - xgboost
  - lightgbm
  - matplotlib
  - seaborn
  - plotly
  - bokeh
  - altair
  - networkx
  - pip
  - pip:
    - tensorflow
    - torch
    - shap
    - optuna
    - linearmodels
    - arch
    - pymc
```

```
    - arviz
```

1.24 23. Reproducibilidad y colaboración vía Git

1.24.1 23.1. Replicar un entorno en otra máquina

En la máquina original:

```
conda activate econblog
conda env export --from-history --format=environment-yaml > environment.yml
git add environment.yml
git commit -m "Add environment configuration"
git push
```

En la máquina nueva:

```
git clone https://github.com/achalmed/epsilon-y-beta.git
cd epsilon-y-beta
conda env create -f environment.yml
conda activate econblog
conda list
```

1.24.2 23.2. Compartir con colaboradores: instrucciones en `README.md`

```
# Configuración del entorno

## Prerrequisitos
- Miniconda o Anaconda instalado
- Git

## Instalación
1. Clonar el repositorio:
   git clone https://github.com/achalmed/epsilon-y-beta.git
   cd epsilon-y-beta

2. Crear el entorno:
   conda env create -f environment.yml

3. Activar el entorno:
   conda activate econblog

4. Verificar instalación:
   python -c "import pandas, statsmodels; print('Entorno listo')"
```

1.24.3 23.3. Actualizar un entorno existente tras cambios en `environment.yml`

```
# Alguien actualizó environment.yml en GitHub
git pull
conda env update -f environment.yml --prune
conda activate econblog
python -c "import pandas, statsmodels; print('Actualización exitosa')"
```

1.25 24. Integración con Jupyter Lab

1.25.1 24.1. Jupyter debe instalarse en cada entorno que lo use

A diferencia de otros gestores, conda no comparte automáticamente Jupyter entre entornos: cada entorno que vaya a usarse desde Jupyter Lab necesita su propia instalación de `jupyterlab` (o, como mínimo, de `ipykernel`).

1.25.2 24.2. Registrar un kernel para un entorno

```
conda activate econblog
python -m ipykernel install --user --name econblog --display-name "Python (econblog)"
jupyter kernelspec list
```

Esto te permite seleccionar "Python (econblog)" como kernel dentro de Jupyter Lab, independientemente de desde qué entorno hayas lanzado el propio Jupyter Lab.

1.25.3 24.3. Iniciar Jupyter Lab desde un entorno específico

```
conda activate econblog
jupyter lab --notebook-dir=~/Proyectos/epsilon-y-beta
```

1.26 25. Integración con VS Code

1.26.1 25.1. Seleccionar el intérprete de conda

Desde la paleta de comandos (`Ctrl+Shift+P`): **"Python: Select Interpreter"**, y elige el intérprete correspondiente al entorno, por ejemplo `Python 3.12.x ('econblog')`.

1.26.2 25.2. Configurar el workspace

`.vscode/settings.json` dentro del proyecto:

```
{
  "python.defaultInterpreterPath": "/home/<usuario>/miniconda3/envs/econblog/bin/pyth
  "python.terminal.activateEnvironment": true
}
```

1.26.3 25.3. Verificar activación en la terminal integrada

```
echo $CONDA_DEFAULT_ENV
```

Debería mostrar el nombre del entorno activo, confirmando que VS Code activó correctamente el entorno configurado.

1.27 26. Mamba como acelerador

`mamba` es una reimplementación del solucionador (*solver*) de dependencias de conda, compatible en sintaxis con `conda` (la mayoría de comandos `conda install`/`conda create` funcionan igual con `mamba install`/`mamba create`), pero significativamente más rápida para resolver entornos complejos. Es ampliamente usado precisamente para los casos de esta guía: entornos con muchos paquetes y dependencias cruzadas (como `econblog` o `datascience`), donde el solver clásico de conda puede tardar considerablemente más.

```
mamba create -n entorno python=3.12 paquete1 paquete2 -y
mamba install paquete -y
mamba update --all -y
```

Para la documentación oficial completa de mamba, consulta mamba.readthedocs.io (ver sección de referencias).

1.28 27. Resolución de problemas comunes

1.28.1 27.1. "Kernel died" en Jupyter

Suele deberse a paquetes incompatibles o falta de memoria. Reinstala el kernel forzando la sobrescritura:

```
conda activate econblog
python -m ipykernel install --user --name econblog --display-name "Python (econblog)"
jupyter kernelspec list
```

Si persiste, recrea el entorno desde un respaldo:

```
conda env export --no-builds > backup-environment.yml
conda deactivate
conda env remove -n econblog -y
conda env create -f backup-environment.yml
```

1.28.2 27.2. `ModuleNotFoundError` en Jupyter pese a tener el paquete instalado

Causa típica: Jupyter está usando el kernel de otro entorno. Verifica el kernel activo en `Kernel > Change kernel`, o reinstala `ipykernel` para el entorno correcto:

```
conda activate econblog
mamba install ipykernel -y
python -m ipykernel install --user --name econblog --display-name "Python (econblog)"
```

1.28.3 27.3. Conflictos de versiones entre paquetes

```
# Opción 1: usar mamba (mejor resolución de dependencias)
mamba install paquete-conflictivo -y

# Opción 2: fijar versiones compatibles explícitamente
mamba install paquete1=1.0 paquete2=2.0 -y

# Opción 3: recrear el entorno limpio desde el environment.yml
conda env remove -n econblog -y
mamba create -n econblog -f environment.yml
```

1.28.4 27.4. Entorno demasiado grande en disco

```
du -sh ~/miniconda3/envs/econblog   # ver tamaño
conda clean --all -y                # limpiar caché de paquetes descargados
```

Si el problema persiste, exporta, edita manualmente el `environment.yml` para quitar paquetes innecesarios, y recrea el entorno:

```
conda env export --no-builds > backup.yml
# editar backup.yml manualmente
conda env remove -n econblog -y
conda env create -f backup.yml
```

1.29 28. Referencia rápida de comandos

```
# ============================================================
# CREAR Y LISTAR
# ============================================================
conda create -n nombre python=3.12 paquetes -y
conda env create -f environment.yml
conda env list                        # o: conda info --envs

# ============================================================
# ACTIVAR / DESACTIVAR
# ============================================================
conda activate nombre
conda deactivate
```

```
# ============================================================
# PAQUETES
# ============================================================
mamba install paquete -y
mamba install paquete=1.0.0 -y
mamba install -c conda-forge paquete -y
mamba update paquete -y
mamba update --all -y
conda remove paquete -y
conda list
pip install paquete   # solo si no está disponible vía conda

# ============================================================
# EXPORTAR / IMPORTAR
# ============================================================
conda export --from-history --format=environment-yaml > environment.yml
conda env export --no-builds > environment.yml          # forma tradicional
conda list --explicit > spec-file.txt                   # reproducción exacta
conda env create -f environment.yml
conda env update -f environment.yml --prune
conda create --name myenv --file spec-file.txt

# ============================================================
# CLONAR / RESTAURAR / ELIMINAR
# ============================================================
conda create --name myclone --clone myenv
conda list --revisions
conda install --rev REVNUM
conda env remove -n nombre -y

# ============================================================
# MANTENIMIENTO
# ============================================================
conda clean --all -y
du -sh ~/miniconda3/envs/*

# ============================================================
# CANALES
# ============================================================
conda config --add channels conda-forge
conda config --set channel_priority strict
```

1.30 39. Referencias oficiales

- **conda — Gestión de entornos (oficial)**: https://docs.conda.io/projects/conda/en/stable/user-guide/tasks/manage-environments.html

- **conda — Gestión de canales (oficial)**: https://docs.conda.io/projects/conda/en/stable/user-guide/tasks/manage-channels.html
- **conda — Gestión de paquetes (oficial)**: https://docs.conda.io/projects/conda/en/stable/user-guide/tasks/manage-pkgs.html
- **conda — Gestión de la propia instalación de conda**: https://docs.conda.io/projects/conda/en/stable/user-guide/tasks/manage-conda.html
- **conda — Archivo de configuración `.condarc`**: https://docs.conda.io/projects/conda/en/stable/user-guide/configuration/use-condarc.html
- **conda — Ajustes de configuración (`settings`)**: https://docs.conda.io/projects/conda/en/stable/user-guide/configuration/settings.html
- **conda — Conceptos: canales**: https://docs.conda.io/projects/conda/en/stable/user-guide/concepts/channels.html
- **conda — Conceptos: entornos**: https://docs.conda.io/projects/conda/en/stable/user-guide/concepts/environments.html
- **conda — Comando `conda create`**: https://docs.conda.io/projects/conda/en/stable/commands/create.html
- **conda — Comando `conda env export`**: https://docs.conda.io/projects/conda/en/stable/commands/env/export.html
- **conda — Hoja de referencia (cheatsheet) oficial**: https://docs.conda.io/projects/conda/en/stable/user-guide/cheatsheet.html
- **conda — Solución de problemas**: https://docs.conda.io/projects/conda/en/stable/user-guide/troubleshooting.html
- **mamba — Documentación oficial**: https://mamba.readthedocs.io/en/latest/user_guide/mamba.html
- **CEP 22 — Frozen environments**: https://conda.org/learn/ceps/cep-0022
- **Jupyter — Documentación oficial**: https://jupyter.org/documentation
- **Publicación previa de referencia**: Achalma, Edison. “Entornos virtuales con Conda”. numerus-scriptum.netlify.app, 2020 (actualizado 2026). https://numerus-scriptum.netlify.app/python/2020-06-20-configurar-entorno-virtual-python-anaconda/

Edison Achalma Economista — Universidad Nacional de San Cristóbal de Huamanga (UNSCH) Ayacucho, Perú ORCID: 0000-0001-6996-3364

2 Publicaciones Similares

Si te interesó este artículo, te recomendamos que explores otros blogs y recursos relacionados que pueden ampliar tus conocimientos. Aquí te dejo algunas sugerencias:

1. Instalacion De Anaconda
2. Configurar Entorno Virtual Python Anaconda
3. 01 Introducion A La Programacion Con Python
4. 02 Variables Expresiones Y Statements Con Python
5. 03 Objetos De Python
6. 04 Ejecucion Condicional Con Python
7. 05 Iteraciones Con Python
8. 06 Funciones Con Python
9. 07 Dataframes Con Python

10. 08 Prediccion Y Metrica De Performance Con Python
11. 09 Metodos De Machine Learning Para Clasificacion Con Python
12. 10 Metodos De Machine Learning Para Regresion Con Python
13. 11 Validacion Cruzada Y Composicion Del Modelo Con Python
14. Visualizacion De Datos Con Python

Esperamos que encuentres estas publicaciones igualmente interesantes y útiles. ¡Disfruta de la lectura!